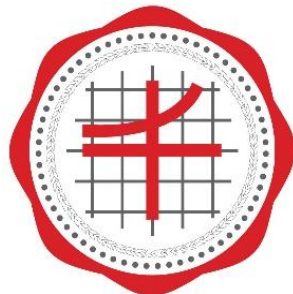


CPE 121 Mobile Application Developments

บทที่ 8

การทำงานกับไฟล์ (File I/O)

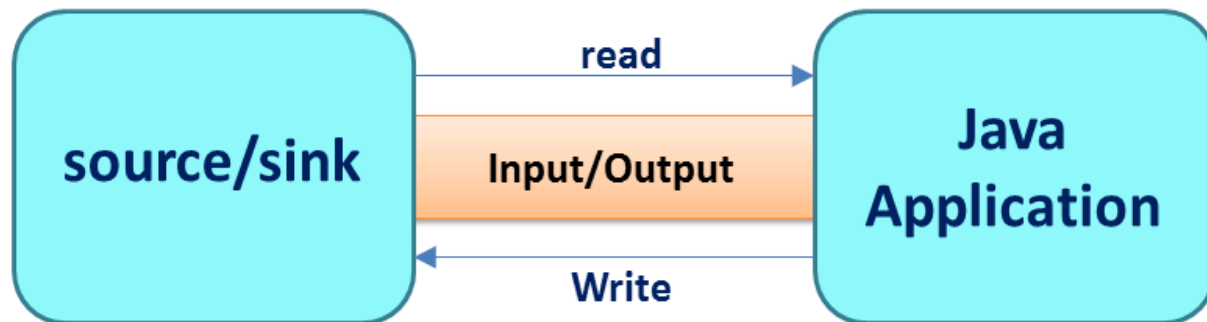


Faculty of Engineering
SRINAKHARINWIROT UNIVERSITY

Pramual Choorat, Ph.D.
Department of Computer Engineering

ไฟล์

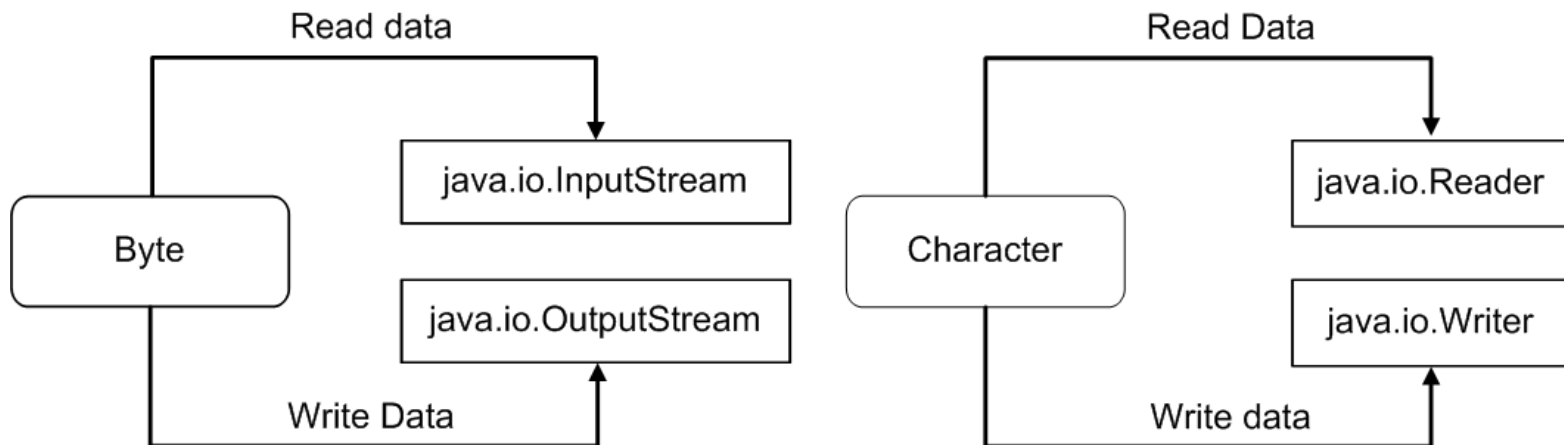
ไฟล์ (File) คือ ข้อมูลต่างๆ ที่ถูกจัดเก็บและบันทึกไว้ในหน่วยความจำของคอมพิวเตอร์ มีหลากหลายประเภท เช่น ไฟล์ข้อมูลเอกสาร ไฟล์รูป เป็นต้น โปรแกรมจะอ่านหรือเขียนข้อมูลเหล่านั้นโดยมีสะพานส่งผ่านข้อมูลอยู่ระหว่างกลาง ใช้ส่งข้อมูลจากต้นทาง (source) ไปยังปลายทาง (sink) และเมื่อมีการอ่านหรือเขียนข้อมูลผ่านสะพานนี้ จะเรียกว่าสตรีม (Stream) และจะถูกนำไปใช้ในการกระบวนกรของโปรแกรมนั่นเอง



ไฟล์สตรีม

ไฟล์สตรีม (File Stream) หมายถึง ท่อส่งข้อมูลไฟล์จากต้นทางไปยังปลายทาง ประกอบด้วยคลาส FileInputStream / FileOutputStream สำหรับการอ่านและเขียนข้อมูล โดยเมธอดของคลาสจะอยู่ในแพ็คเกจ java.io และมีคลาสสำหรับการสร้างออบเจกต์ Stream 4 คลาส ได้แก่

- ❖ **InputStream** ใช้ในการรับชนิดข้อมูลแบบ byte หรือ numeric type
- ❖ **OutputStream** ใช้ในการส่งชนิดข้อมูลแบบ byte หรือ numeric type
- ❖ **Reader** ใช้ในการรับชนิดข้อมูลแบบ character type หรือ String
- ❖ **Writer** ใช้ในการส่งชนิดข้อมูลแบบ character type หรือ String



การรับส่งข้อมูลแบบ Byte Stream

การรับส่งข้อมูลแบบ **Byte Stream** เป็นการเขียนและอ่านไฟล์ครั้งละ 1 ไบต์ (8 บิต) ใช้สำหรับรับส่งข้อมูลประเภท **Byte** หรือข้อมูลประเภทตัวเลข (Numeric) โดยจะต้องทำผ่านคลาส `FileOutputStream` และ `FileInputStream` ซึ่งอยู่ในแพ็คเกจ `java.io.OutputStream` และ `java.io.InputStream` ตามลำดับ ข้อมูลที่ได้ถูกจัดเก็บเป็นไบนารี (Binary File) และไม่สามารถจะอ่านเข้าใจได้ จะใช้งานคลาส `File` ร่วมกับคลาส `FileOutputStream` ในการเขียนข้อมูลลงไฟล์ โดยใช้เมธอด `write()` ของคลาส `FileOutputStream` มีรูปแบบดังนี้

```
File filename = new File("path");
```

```
FileOutputStream fOutput = new FileOutputStream(filename, append);  
fOutput.write(data);
```

หรือ

```
FileOutputStream fOutput = new FileOutputStream(path, append);  
fOutput.write(data);
```

โดยที่

path

filename

fOutput

append

data

เป็นชื่อและตำแหน่งที่อยู่ของไฟล์

เป็นออบเจกต์ที่สร้างจากคลาส `File`

เป็นออบเจกต์ที่สร้างจากคลาส `FileOutputStream`

มีค่าเป็น `true` เมื่อต้องการเขียนข้อมูลต่อท้ายข้อมูลในไฟล์เดิม

เป็นข้อมูลที่ต้องการเขียนลงไฟล์

การรับส่งข้อมูลแบบ Byte Stream

```
File filename = new File("d:\\ByteData.txt");
FileOutputStream f_Output = null;
try{
    int c = 10;
    f_Output = new FileOutputStream(filename , true);
    f_Output .write(c);
}
catch (IOException e){
    System.out.println(e);
}
```



การอ่านข้อมูลแบบ Byte Stream

การอ่านข้อมูลแบบ **Byte Stream** จะเป็นการใช้งานคลาส **File** ร่วมกับคลาส **FileInputStream** ในการอ่านข้อมูลจากไฟล์ ใช้เมธอด **read()** ของคลาส **FileInputStream** และจะคืนค่าเป็นเลขจำนวนเต็ม -1 เมื่ออ่านข้อมูลถึงไบต์สุดท้ายของไฟล์ มีรูปแบบ ดังนี้

```
File filename = new File("path");
FileInputStream fInput = new FileInputStream (filename);
data = fInput.read();
หรือ
FileInputStream fOutput = new FileInputStream (path);
data = fOutput.read();
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
fInput	เป็นออบเจกต์ที่สร้างจากคลาส FileInputStream
data	เป็นข้อมูลที่อ่านได้จากไฟล์



การอ่านข้อมูลแบบ Byte Stream

```
File filename = new File ("d:\\ByteData.txt");
FileInputStream f_in = null;
try{
    f_in = new FileInputStream(filename);
    int c;
    while ((c = f_in.read()) != -1)
        System.out.println(c)
}
catch (IOException e){
    System.out.println(e);
}
```



โปรแกรมเขียนและอ่านข้อมูลตัวเลขแบบ Byte Stream

```
import java.io.*;
import java.util.Scanner;
public class ByteTest {
    public static void main(String[] args) {
        String fname = "d:\\ByteData.txt";    FileOutputStream f_out = null;
        try {
            f_out = new FileOutputStream(fname);
            int c;
            System.out.println("Read from KB");
            System.out.println("=====");
            do {
                System.out.print("Enter Data : ");
                c = new Scanner(System.in).nextInt();
                if (c != 0) f_out.write(c);
            } while (c != 0);
        }
        catch (IOException e){
            System.out.println(e);
        }
    }
}
```

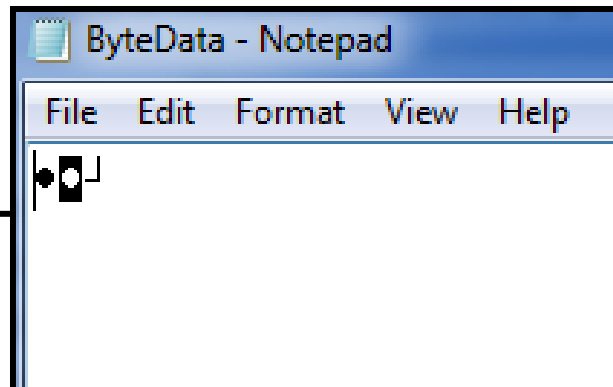
โปรแกรมเขียนและอ่านข้อมูลตัวเลขแบบ Byte Stream

```
finally {
    try {
        if (f_out != null)
            f_out.close();
        System.out.println("Data are saved to File : \n" + fname);
        System.out.println("=====");
    }
    catch (IOException e){
        System.out.println(e);
    }
}

FileInputStream f_in = null;
try {
    f_in = new FileInputStream(fname);
    System.out.println("Read Data from File : \n" + fname);
    System.out.println("=====");
    while ((c = f_in.read()) != -1)
        System.out.printf("%-3d", c);
    System.out.println(); }
    int c;
```

โปรแกรมเขียนและอ่านข้อมูลตัวเลขแบบ Byte Stream

```
catch (IOException e){
    System.out.println(e);
}
finally {
    try {
        if (f_in != null)
            f_in.close();
        System.out.println("=====");
    }
    catch (IOException e) {
        System.out.println(e);
    }
}
}
```



```
Read from KB
=====
Enter Data : 7
Enter Data : 8
Enter Data : 4
Enter Data : 0

Data are saved to File :
d:\ByteData.txt
=====
Read Data from File :
d:\ByteData.txt
=====
```

การรับส่งข้อมูลแบบ Character Stream

การรับส่งข้อมูลแบบ **Character Stream** เป็นการอ่านและเขียนไฟล์ครั้งละ 2 ไบต์ (16 บิต) ใช้สำหรับรับส่งข้อมูลประเภทตัวอักษร (Character) จัดการโดยใช้คลาส `FileReader` และ `FileWriter` ซึ่งอยู่ในแพ็คเกจ `java.io.Reader` และ `java.io.Writer` ตามลำดับ โดยข้อมูลในไฟล์จะถูกจัดเก็บเป็นกลุ่มตัวอักษร (Text File) ซึ่งสามารถอ่านเข้าใจได้ จะใช้งานคลาส `File` ร่วมกับคลาส `FileWriter` ในการเขียนข้อมูลลงไฟล์ โดยใช้เมธอด `write()` ของคลาส `FileWriter` มีรูปแบบ ดังนี้

```
File filename = new File("path");  
FileWriter fOutput = new FileWriter(filename, append);  
fOutput.write(data);  
หรือ  
FileWriter fOutput = new FileWriter(path, append);  
fOutput.write(data);
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส <code>File</code>
fOutput	เป็นออบเจกต์ที่สร้างจากคลาส <code>FileWriter</code>
append	มีค่าเป็น <code>true</code> เมื่อต้องการเขียนข้อมูลต่อท้ายข้อมูลในไฟล์เดิม
data	เป็นข้อมูลที่ต้องการเขียนลงไฟล์

การเขียนข้อมูลแบบ Character Stream

```
File filename = new File("d:\\CharData.txt");
FileWriter f_write = null;
try{
    String c = "*";
    f_write = new FileWriter(filename, true);
    f_write.write(c)
}
catch (IOException e){
    System.out.println(e);
}
```



การอ่านข้อมูลแบบ Character Stream

การอ่านข้อมูลแบบ **Character Stream** จะเป็นการใช้งานคลาส **File** ร่วมกับคลาส **FileReader** ในการอ่านข้อมูลจากไฟล์ โดยใช้เมธอด **read()** ของคลาส **FileReader** และจะคืนค่าเป็นเลขจำนวนเต็ม **-1** เมื่อสิ้นสุดการอ่านข้อมูลจากไฟล์ มีรูปแบบ ดังนี้

```
File filename = new File("path");
FileReader fInput = new FileReader (filename);
data = fInput.read();
หรือ
FileReader fOutput = new FileReader (path);
data = fOutput.read();
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
fInput	เป็นออบเจกต์ที่สร้างจากคลาส FileReader
data	เป็นข้อมูลที่อ่านได้จากไฟล์



การอ่านข้อมูลแบบ Character Stream

```
File filename = new File("d:\\CharData.txt");
FileReader f_read = null;
try{
    f_read = new FileReader(filename);
    int c ;
    while ((c = f_read.read()) != -1)
        System.out.println(c);
}
catch (IOException e){
    System.out.println(e);
}
```



โปรแกรมเขียนและอ่านข้อมูลตัวอักขระแบบ Character Stream

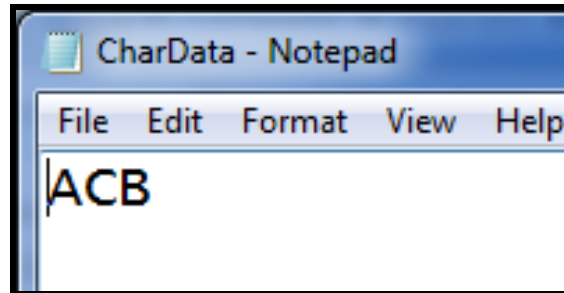
```
import java.io.*;
import java.util.Scanner;
public class CharTest {
    public static void main(String[] args) {
        String fname = "d:\\CharData.txt";
        FileWriter f_write = null;
        try {
            f_write = new FileWriter(fname);
            String c;
            System.out.println("Read from KB");
            System.out.println("=====");
            do {
                System.out.print("Enter Data : ");
                c = new Scanner(System.in).nextLine();
                if (!(c.equals("*"))) f_write.write(c);
            } while (!(c.equals("*")));
        } catch (IOException e){
            System.out.println(e);
        }
    }
}
```

โปรแกรมเขียนและอ่านข้อมูลตัวอักขระแบบ Character Stream

```
finally {
    try {
        if (f_write != null)
            f_write.close();
        System.out.println("Data are saved to File : \n" + fname);
        System.out.println("=====");
    } catch (IOException e){
        System.out.println(e); }
}
FileReader f_read = null;
try {
    f_read = new FileReader(fname);
    System.out.println("Read Data from File : \n" + fname);
    System.out.println("=====");
    while ((c = f_read.read()) != -1) {
        System.out.print(c);
        System.out.print("\t");
    }
    System.out.println(); }
```

โปรแกรมเขียนและอ่านข้อมูลตัวอักขระแบบ Character Stream

```
catch (IOException e){
    System.out.println(e);
}
finally {
    try {
        if (f_read != null)
            f_read.close();
        System.out.println("=====");
    }
    catch (IOException e){
        System.out.println(e);
    }
}
}
```



```
Read from KB
=====
Enter Data : A
Enter Data : C
Enter Data : B
Enter Data : *
Data are saved to File :
d:\CharData.txt
=====
Read Data from File :
d:\CharData.txt
=====
65      67      66
=====
```

ผลที่ได้จากการใช้เมธอด print() จะเป็นค่าตัวเลขรหัสแอสกีของตัวอักขระ และสามารถใช้เมธอด write() แสดงผลข้อมูลเป็นตัวอักขระได้



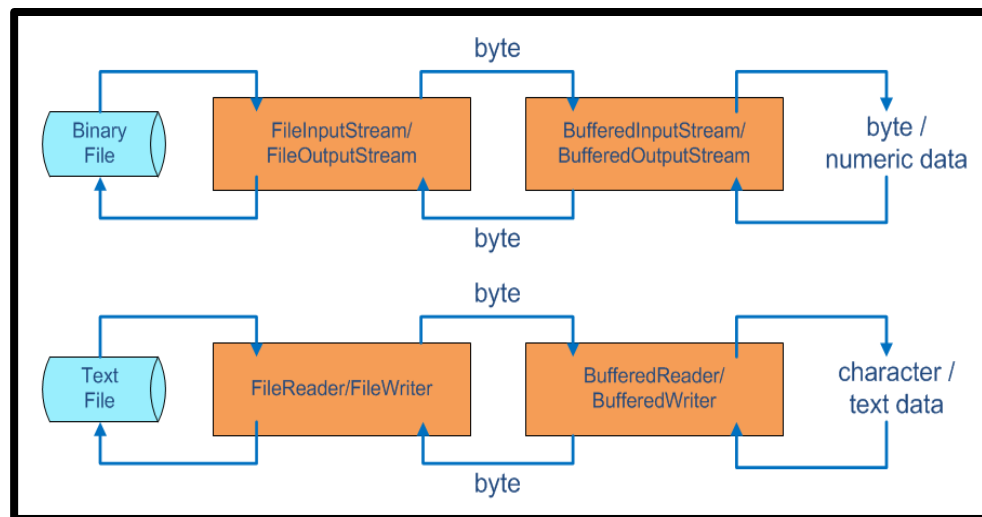
การรับส่งข้อมูลด้วย Buffer Stream

Buffer Stream เป็นที่ปักในหน่วยความจำในการรับส่งข้อมูลผ่าน buffer ซึ่งเป็นการนำข้อมูลที่ต้องการอ่านหรือเขียนลงไฟล์ไปเก็บไว้ในหน่วยความจำก่อน เพื่อช่วยเพิ่มประสิทธิภาพให้โปรแกรมมากขึ้น ทำงานเร็วขึ้น

คลาส Buffer ของ **Byte Stream** คือคลาส **BufferedInputStream** และ **BufferedOutputStream**

ส่วนคลาส Buffer ของ **Character Stream** คือ คลาส **BufferedReader** และ **BufferedWriter**

การใช้งานคลาส buffer จะต้องสร้างออบเจกต์เพื่อเชื่อมต่อออบเจกต์ของ คลาส FileInputStream กับ FileOutputStream หรือ FileReader กับ FileWriter



การเขียนข้อมูลแบบ Buffer Stream ด้วย Buffer Stream

รูปแบบการใช้งานคลาส **BufferedOutputStream** สำหรับเขียนข้อมูลแบบ Byte Stream มีดังนี้

```
File filename = new File(Path);  
FileOutputStream f_out = new FileOutputStream(filename);  
BufferedOutputStream b_out = new BufferedOutputStream(f_out);  
b_out.write(data);
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
f_out	เป็นออบเจกต์ที่สร้างจากคลาส FileOutputStream
b_out	เป็นออบเจกต์ที่สร้างจากคลาส BufferedOutputStream
data	เป็นข้อมูลที่ต้องการเขียนลงไฟล์



การเขียนข้อมูลแบบ Buffer Stream ด้วย Buffer Stream

```
File filename = new File("d:\\ByteBuffData.txt");
FileOutputStream f_out = null;
BufferedOutputStream b_out = null;
try{
    f_out = new FileOutputStream(filename);
    b_out = new BufferedOutputStream(f_out);
    int c = 10;
    b_out.write(c);
}
catch (IOException e){
    System.out.println(e);
}
```



การอ่านข้อมูลแบบ Buffer Stream ด้วย Buffer Stream

รูปแบบการใช้งานคลาส **BufferedInputStream** สำหรับเขียนข้อมูลแบบ Byte Stream มีดังนี้

```
File filename = new File(Path);  
FileInputStream f_in = new FileInputStream(filename);  
BufferedInputStream b_in = new BufferedInputStream(f_in);  
data = b_in.read();
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
f_in	เป็นออบเจกต์ที่สร้างจากคลาส FileOutputStream
b_in	เป็นออบเจกต์ที่สร้างจากคลาส BufferedOutputStream
data	เป็นข้อมูลที่อ่านได้จากไฟล์



การอ่านข้อมูลแบบ Buffer Stream ด้วย Buffer Stream

```
File filename = new File("d:\\ByteBuffData.txt");
FileInputStream f_in = null;
BufferedInputStream b_in = null;
try{
    f_in = new FileInputStream(filename);
    b_in = new BufferedInputStream(f_in);
    int c ;
    while ((c = b_in.read()) != -1)
        System.out.println(c);
}
catch (IOException e){
    System.out.println(e);
}
```



โปรแกรมเขียนและอ่านข้อมูลตัวเลขแบบ Byte Stream ผ่าน buffer

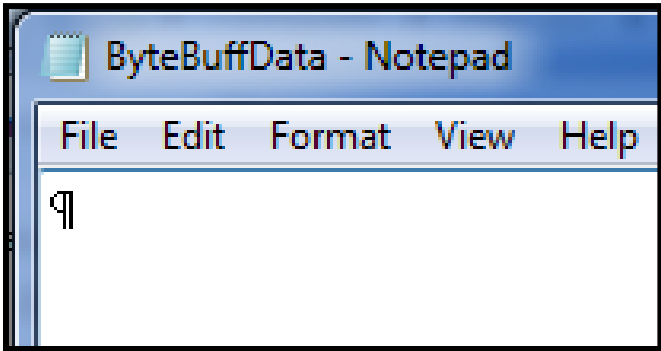
```
import java.io.*;
import java.util.Scanner;
public class ByteBuffTest {
    public static void main(String[] args) {
        String fname = "d:\\ByteBuffData.txt";
        FileOutputStream f_out = null;
        BufferedOutputStream b_out = null;
        try {
            f_out = new FileOutputStream(fname);
            b_out = new BufferedOutputStream(f_out);
            int c;
            System.out.println("Read from KB");
            System.out.println("=====");
            do {
                System.out.print("Enter Data : ");
                c = new Scanner(System.in).nextInt();
                if (c != 0) b_out.write(c);
            } while (c != 0);
        }
    }
}
```

โปรแกรมเขียนและอ่านข้อมูลตัวเลขแบบ Byte Stream ผ่าน buffer

```
catch (IOException e){
    System.out.println(e); }
finally {
    try {
        if (b_out != null)
            b_out.close();
        System.out.println("Data are saved to File : \n" + fname);
        System.out.println("=====");    }
    catch (IOException e){
        System.out.println(e); }
}
FileInputStream f_in = null;
BufferedInputStream b_in = null;
try {
    f_in = new FileInputStream(fname);
    b_in = new BufferedInputStream(f_in);
    System.out.println("Read Data from File : \n" + fname);
    System.out.println("=====");
    int c;
```

โปรแกรมเขียนและอ่านข้อมูลตัวเลขแบบ Byte Stream ผ่าน buffer

```
while ((c = b_in.read()) != -1)
    System.out.printf("%-3d",c);
    System.out.println();
}
catch (IOException e){
    System.out.println(e);
}
finally {
    try {
        if (b_in != null)
            b_in.close();
        System.out.println("=====");
    }
    catch (IOException e) {
        System.out.println(e);
    }
}
}
```



```
Read from KB
=====
Enter Data : 10
Enter Data : 20
Enter Data : 30
Enter Data : 0

Data are saved to File :
d:\ByteBuffData.txt
=====
Read Data from File :
d:\ByteBuffData.txt
=====
10 20 30
=====
```

การเขียนข้อมูลแบบ Character Stream ด้วย BufferedWriter

คลาส **BufferedWriter** มีรูปแบบการใช้งานสำหรับเขียนข้อมูลแบบ Character Stream ดังนี้

```
File filename = new File(Path);  
FileWriter f_write = new FileWriter(filename);  
BufferedWriter b_write = new BufferedWriter(f_write);  
b_write.write(data);
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
f_write	เป็นออบเจกต์ที่สร้างจากคลาส FileWriter
b_write	เป็นออบเจกต์ที่สร้างจากคลาส BufferedWriter
data	เป็นข้อมูลที่ต้องการเขียนลงไฟล์



การเขียนข้อมูลแบบ Character Stream ด้วย BufferedWriter

```
File filename = new File("d:\\CharBuffData.txt");
FileWriter f_write = null;
BufferedWriter b_write = null;
try{
    String msg = "Java"+"\\n"+"Programming";
    f_write = new FileWriter(filename);
    b_write = new BufferedWriter(f_write);
    b_write.write(msg);
}
catch (IOException e){
    System.out.println(e);
}
```



การอ่านข้อมูลแบบ Character Stream ด้วย BufferedWriter

คลาส **BufferedReader** มีรูปแบบการใช้งานสำหรับอ่านข้อมูลแบบ **Character Stream** ดังนี้

```
File filename = new File(Path);  
FileReader f_read = new FileReader(filename);  
BufferedReader b_read = new BufferedReader(f_read);  
data = b_read.readLine();
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
f_read	เป็นออบเจกต์ที่สร้างจากคลาส FileReader
b_read	เป็นออบเจกต์ที่สร้างจากคลาส BufferedReader
data	เป็นข้อมูลที่อ่านได้จากไฟล์



โปรแกรมเขียนและอ่านข้อมูลตัวอักขระแบบ Character Stream ผ่าน buffer

```
import java.io.*;
import java.util.Scanner;
public class CharBuffTest {
    public static void main(String[] args) {
        String fname = "d:\\CharBuffData.txt";
        FileWriter f_write = null;
        BufferedWriter b_write = null;
        try {
            f_write = new FileWriter(fname);
            b_write = new BufferedWriter(f_write);
            String c;
            System.out.println("Read from KB");
            System.out.println("=====");
            do {
                System.out.print("Enter Data : ");
                c = new Scanner(System.in).nextLine();
                if (!(c.equals("*"))) b_write.write(c+"\n");
            } while (!(c.equals("*")));
        }
    }
}
```

โปรแกรมเขียนและอ่านข้อมูลตัวอักขระแบบ Character Stream ผ่าน buffer

```
catch (IOException e){
    System.out.println(e);  }
finally {
    try {
        if (b_write != null)
            b_write.close();
        System.out.println("Data are saved to File : \n" + fname);
        System.out.println("=====");    }
    catch (IOException e){
        System.out.println(e);    }
}

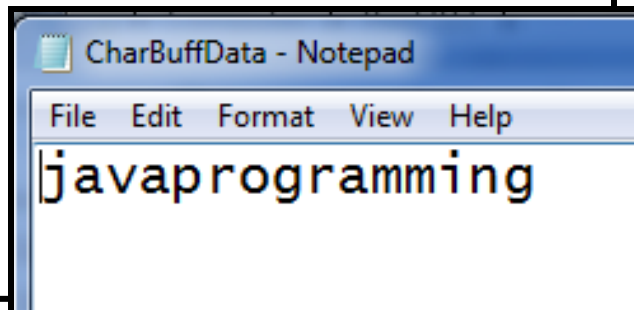
FileReader f_read = null;
BufferedReader b_read = null;
try {
    f_read = new FileReader(fname);
    b_read = new BufferedReader(f_read);
    System.out.println("Read Data from File : \n" + fname);
    System.out.println("=====");
    String c;
```

โปรแกรมเขียนและอ่านข้อมูลตัวอักขระแบบ Character Stream ผ่าน buffer

```
while ((c = b_read.readLine()) != null) {
    System.out.print(c);
    System.out.print("\t");    }
    System.out.println();
}
catch (IOException e){
    System.out.println(e);
}
finally {
    try {
        if (b_read != null)
            b_read.close();
        System.out.println("=====");
    }
    catch (IOException e){
        System.out.println(e);
    }
}
```

```
Read from KB
=====
Enter Data : java
Enter Data : programming
Enter Data : *

Data are saved to File :
d:\CharBuffData.txt
=====
Read Data from File :
d:\CharBuffData.txt
=====
java      programming
=====
```



โปรแกรมเขียนและอ่านข้อมูลแบบ Character Stream ผ่าน buffer โดยใช้เมธอด skip()

```
import java.io.*;
import java.util.Scanner;
public class CharSkipTest {
    public static void main(String[] args) {
        String fname = "d:\\CharBuffData.txt";
        FileWriter f_write = null;
        BufferedWriter b_write = null;
        try {
            f_write = new FileWriter(fname);
            b_write = new BufferedWriter(f_write);
            String c;
            System.out.println("Read from KB");
            System.out.println("=====");
            do {
                System.out.print("Enter Data : ");
                c = new Scanner(System.in).nextLine();
                if (!(c.equals("*"))) b_write.write(c);
            } while (!(c.equals("*")));
        }
    }
}
```

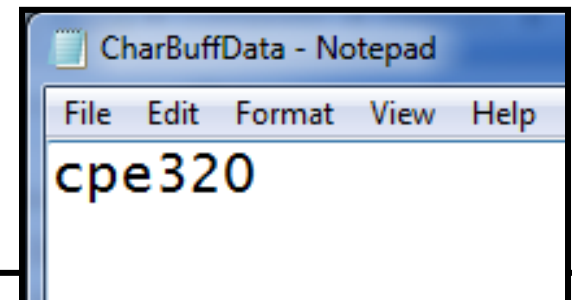
โปรแกรมเขียนและอ่านข้อมูลแบบ Character Stream ผ่าน buffer โดยใช้เมธอด skip()

```
catch (IOException e){
    System.out.println(e);  }
finally {
    try {
        if (b_write != null)
            b_write.close();
        System.out.println("Data are saved to File : \n" + fname);
        System.out.println("=====");
    }
    catch (IOException e){
        System.out.println(e);    }
}
FileReader f_read = null;
BufferedReader b_read = null;
try {
    f_read = new FileReader(fname);
    b_read = new BufferedReader(f_read);
    System.out.println("Read Data from File : \n" + fname);
    System.out.println("=====");    int c;
```

โปรแกรมเขียนและอ่านข้อมูลแบบ Character Stream ผ่าน buffer โดยใช้เมธอด skip()

```
while ((c = b_read.read()) != -1) {  
    System.out.write(c);  
    b_read.skip(2);  
    System.out.print("\t");  
}  
System.out.println();  
}  
catch (IOException e){  
    System.out.println(e); }  
finally {  
    try {  
        if (b_read != null)  
            b_read.close();  
        System.out.println("=====");  
    }  
    catch (IOException e){  
        System.out.println(e); }  
}
```

```
Read from KB  
=====  
Enter Data : cpe320  
Enter Data : *  
Data are saved to File :  
d:\CharBuffData.txt  
=====  
Read Data from File :  
d:\CharBuffData.txt  
=====  
c          3  
=====
```



การเขียนข้อมูลด้วยคลาส **PrintWriter**

การเขียนข้อมูลด้วยคลาส **PrintWriter** มีการใช้เมธอด **print()** และ **println()** ซึ่งมีการโอเวอร์โหลดให้รับค่าข้อมูลที่เป็นชนิดพื้นฐานได้ทั้งหมด โดยมีรูปแบบการใช้งานคลาส **PrintWriter** ดังนี้

```
File filename = new File(path);  
FileWriter f_write = new FileWriter(filename);  
BufferedWriter b_write = new BufferedWriter(f_write);  
PrintWriter p_write = new PrintWriter(b_write);  
p_write.println(data);
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
f_write	เป็นออบเจกต์ที่สร้างจากคลาส FileWriter
b_write	เป็นออบเจกต์ที่สร้างจากคลาส BufferedWriter
p_write	เป็นออบเจกต์ที่สร้างจากคลาส PrintWriter
data	เป็นข้อมูลที่ต้องการเขียนลงไฟล์



การเขียนข้อมูลด้วยคลาส **PrintWriter**

```
File filename = new File("d:\\WriteCharData.txt");
FileWriter f_write = null;
BufferedWriter b_write = null;
PrintWriter p_write = null;
int id = 1; String name = "Java";
float rating = 4.5f;
try{
    f_write = new FileWriter(filename);
    b_write = new BufferedWriter(f_write);
    p_write = new PrintWriter(b_write);
    p_write.println(id+name+rating);
}
catch (IOException e){
    System.out.println(e);
}
```

โปรแกรมเขียนข้อมูลที่มีชนิดแตกต่างกันแบบ Character Stream

การจัดเก็บข้อมูลที่ประกอบด้วยชนิดข้อมูลแตกต่างกัน ควรใช้การอ่านและเขียนข้อมูลลงไฟล์ประเภท Binary File แต่ถ้าต้องการทำด้วย Character Stream ก็สามารถทำได้โดยการค้นข้อมูลแต่ละชนิดที่จะเขียนลงไฟล์ด้วย “\t” CharWriteTest เขียนข้อมูลลงไฟล์ CharScore.txt



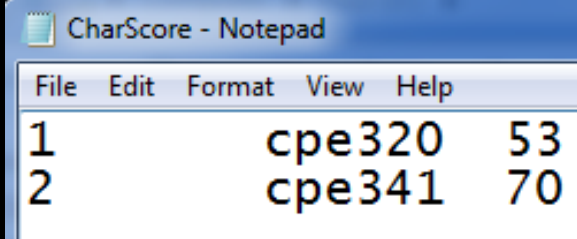
โปรแกรมเขียนข้อมูลที่มีชนิดแตกต่างกันแบบ Character Stream

```
import java.io.*;
import java.util.Scanner;
public class CharWriteTest {
    public static void main(String[] args) {
        int id; String name, score;
        File ofile = new File("d:\\CharScore.txt");
        FileWriter f_write = null;
        BufferedWriter b_write = null;
        PrintWriter p_write = null;
        try {
            f_write = new FileWriter(ofile);
            b_write = new BufferedWriter(f_write);
            p_write = new PrintWriter(b_write);
            System.out.println("Read from KB");
            System.out.println("=====");
            do {
                System.out.print("id = ");
                id = new Scanner(System.in).nextInt();
                if (id != -1) {
```

โปรแกรมเขียนข้อมูลที่มีชนิดแตกต่างกันแบบ Character Stream

```
System.out.print("name = ");
name = new Scanner(System.in).nextLine();
System.out.print("score = ");
score = new Scanner(System.in).nextLine();
p_write.println(id+"\t"+name+"\t"+score);
}
} while (id != -1);
}
catch (IOException e){
    System.out.println(e);
}
finally {
    if (p_write != null)
        p_write.close();
    System.out.println("Data are saved to File : \n" + ofile);
    System.out.println("=====");
}
}
```

```
Read from KB
=====
id = 1
name = cpe320
score = 53
id = 2
name = cpe341
score = 70
id = -1
Data are saved to File :
d:\CharScore.txt
=====
```



File	Edit	Format	View	Help
1		cpe320	53	
2		cpe341	70	

โปรแกรมอ่านและแสดงผลข้อมูลจากไฟล์ d:\\CharScore.txt

```
import java.io.*;
class std {
    int i;
    String n;
    double s;
}
public class CharReadTest {
    public static void main(String[] args) {
        File ifile = new File("d:\\CharScore.txt");
        FileReader f_read = null;
        BufferedReader b_read = null;
        std[] x = new std[20];
        double sum=0; int c=0;
        String line;
        try {
            f_read = new FileReader(ifile);
            b_read = new BufferedReader(f_read);
            System.out.println("Read Data from File : \\n" + ifile);
            System.out.println("=====");
```

โปรแกรมอ่านและแสดงผลข้อมูลจากไฟล์ d:\CharScore.txt

```
while ((line = b_read.readLine()) != null) {
    x[c] = new std();
    String msg[] = line.split("\\t",3);
    x[c].i = Integer.parseInt(msg[0]);
    x[c].n = msg[1];
    x[c].s = Double.parseDouble(msg[2]);
    sum = sum+x[c].s;
    c++;
}
}
catch (IOException e){
    System.out.println(e);
}
finally {
    try {
        if (b_read != null)
            b_read.close();
        double avg =sum/c;
```

โปรแกรมอ่านและแสดงผลข้อมูลจากไฟล์ d:\CharScore.txt

```
for (int i=0;i<c;i++)
    System.out.printf("%-3d%-10s%-6.2f\n",x[i].i,x[i].n,x[i].s);
    System.out.println("=====");
    System.out.printf("mean = %.2f\n",avg);
    System.out.println("=====");
}
catch (IOException e){
    System.out.println(e);
}
}
```

```
Read Data from File :
d:\CharScore.txt
=====
1   cpe320    53.00
2   cpe341    70.00
=====
mean = 61.50
=====
```



คลาส DataOutputStream

คลาส **DataOutputStream** มีรูปแบบการใช้งาน ดังนี้

```
File filename = new File(path);
FileOutputStream f_out = new FileOutputStream(filename);
BufferedOutputStream b_out = new BufferedOutputStream(f_out);
DataOutputStream d_out = new DataOutputStream(b_out);
d_out.methodName(data);
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
f_out	เป็นออบเจกต์ที่สร้างจากคลาส FileOutputStream
b_out	เป็นออบเจกต์ที่สร้างจากคลาส BufferedOutputStream
d_out	เป็นออบเจกต์ที่สร้างจากคลาส DataOutputStream
methodName	เป็นชื่อเมธอดที่ต้องการใช้งาน
data	เป็นข้อมูลที่ต้องการเขียนลงไฟล์



การใช้งานคลาส `DataOutputStream`

```
File filename = new File("d:\\ByteScore.txt");
FileOutputStream f_out = null;
BufferedOutputStream b_out = null;
DataOutputStream d_out = null;
int id = 1; String name = "Java";
float rating = 4.5f;
try{
    f_out = new FileOutputStream(filename);
    b_out = new BufferedOutputStream(f_out);
    d_out = new DataOutputStream(b_out);
    d_out.writeInt(id);
    d_out.writeUTF(name);
    d_out.writeFloat(rating);
}
catch (IOException e){
    System.out.println(e);
}
```

คลาส DataInputStream

คลาส **DataOutputStream** มีรูปแบบการใช้งาน ดังนี้

```
File filename = new File(Path);  
FileInputStream f_in = new FileInputStream(filename);  
BufferedInputStream b_in = new BufferedInputStream(f_in);  
DataInputStream d_in = new DataInputStream(b_in);  
data = d_in.methodName();
```

โดยที่

path	เป็นชื่อและตำแหน่งที่อยู่ของไฟล์
filename	เป็นออบเจกต์ที่สร้างจากคลาส File
f_in	เป็นออบเจกต์ที่สร้างจากคลาส FileInputStream
b_in	เป็นออบเจกต์ที่สร้างจากคลาส BufferedInputStream
d_in	เป็นออบเจกต์ที่สร้างจากคลาส DataInputStream
methodName	เป็นชื่อเมธอดที่ต้องการใช้งาน
data	เป็นข้อมูลที่อ่านได้จากไฟล์



การใช้งานคลาส DataInputStream

```
File filename = new File("d:\\ByteScore.txt");
FileInputStream f_in = null;
BufferedInputStream b_in = null;
DataInputStream d_in = null;
boolean eof = false;
try{
    f_in = new FileInputStream(filename);
    b_in = new BufferedInputStream(f_in);
    d_in = new DataInputStream(b_in);
    while (!eof){
        System.out.println(d_in.readInt());
        System.out.println(d_in.readUTF());
        System.out.println(d_in.readFloat());
    }
}
```

```
catch (EOFException e){
    eof = true;
}
catch (IOException e){
    System.out.println(e);
}
```



เมธอดที่ควรรู้ในคลาส `DataInputStream`

ชื่อเมธอด	คำอธิบาย
<code>readBoolean()</code>	อ่านค่าข้อมูลชนิด boolean
<code>readByte()</code>	อ่านค่าข้อมูลชนิด byte
<code>readUnsignedByte()</code>	อ่านค่าข้อมูลชนิด byte แบบ unsigned (ไม่มีค่าติดลบ)
<code>readInt()</code>	อ่านค่าข้อมูลชนิด int
<code>readShort()</code>	อ่านค่าข้อมูลชนิด short
<code>readUnsignedShort()</code>	อ่านค่าข้อมูลชนิด short แบบ unsigned (ไม่มีค่าติดลบ)
<code>readLong()</code>	อ่านค่าข้อมูลชนิด long
<code>readFloat()</code>	อ่านค่าข้อมูลชนิด float
<code>readDouble()</code>	อ่านค่าข้อมูลชนิด double
<code>readChar()</code>	อ่านค่าข้อมูลชนิด char
<code>readUTF()</code>	อ่านค่าข้อมูลชนิด String จากไฟล์ในรูปของตัวอักษรทีละตัวเรียงต่อกัน (โดยถอดรหัสในรูปของ UTF)



เมธอดที่ควรรู้ในคลาส **DataInputStream**

ชื่อเมธอด	คำอธิบาย
<code>writeBoolean(boolean)</code>	เขียนข้อมูลชนิด boolean (1 byte) ลงในไฟล์
<code>writeByte(int)</code>	เขียนข้อมูลชนิด byte (1 byte ของ int ที่ส่งมา) ลงในไฟล์
<code>writeInt(int)</code>	เขียนข้อมูลชนิด int (4 byte) ลงในไฟล์
<code>writeShort(int)</code>	เขียนข้อมูลชนิด short (ข้อมูล 2 byte) ลงในไฟล์
<code>writeLong(long)</code>	เขียนข้อมูลชนิด long (ข้อมูล 8 byte) ลงในไฟล์
<code>writeFloat(float)</code>	เขียนข้อมูลชนิด float ลงในไฟล์ โดย float ที่ส่งมาจะถูกแปลงไปเป็น int ด้วยเมธอด <code>floatToIntBits</code> ของคลาส <code>Float</code>
<code>writeDouble(double)</code>	เขียนข้อมูลชนิด double ลงในไฟล์ โดย double ที่ส่งมาจะถูกแปลงไปเป็น long ด้วยเมธอด <code>doubleToLongBits</code> ของคลาส <code>Double</code>
<code>writeChar(int)</code>	เขียนข้อมูลชนิด int (หมายถึงเขียนข้อมูลชนิด 2-byte) ลงในไฟล์
<code>writeChars(String)</code>	เขียนข้อมูลชนิด String ลงในไฟล์ ในรูปของตัวอักษรทีละตัวเรียงต่อกัน
<code>writeUTF(String)</code>	เขียนข้อมูลชนิด String ลงในไฟล์ ในรูปของตัวอักษรทีละตัวเรียงต่อกัน



โปรแกรมเขียนข้อมูลที่มีชนิดแตกต่างกัน แบบ Byte Stream

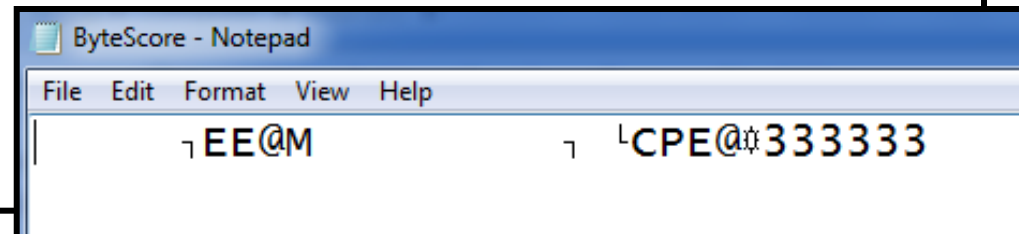
```
import java.util.Scanner;
import java.io.*;
public class ByteDataOutputTest {
    public static void main(String[] args) {
        int id; String name; double score;
        File fname = new File("d:\\ByteScore.txt");
        FileOutputStream f_out = null;
        BufferedOutputStream b_out = null;
        DataOutputStream d_out = null;
        System.out.println("Read Data from File : \n" + fname);
        System.out.println("=====");
        try {
            f_out = new FileOutputStream(fname);
            b_out = new BufferedOutputStream(f_out);
            d_out = new DataOutputStream(b_out);
            do {
                System.out.print("Enter id : ");
                id = new Scanner(System.in).nextInt();
```

โปรแกรมเขียนข้อมูลที่มีชนิดแตกต่างกัน แบบ Byte Stream

```
if (id != -1) {  
    System.out.print("Enter name : ");  
    name = new Scanner(System.in).nextLine();  
    System.out.print("Enter score : ");  
    score = new Scanner(System.in).nextDouble();  
    d_out.writeInt(id);  
    d_out.writeUTF(name);  
    d_out.writeDouble(score);  
} while (id != -1);  
catch (IOException e){  
    System.out.println(e);  
} finally {  
    try {  
        if (d_out != null)  
            d_out.close();  
    } catch (IOException e){  
        System.out.println(e);  
    }  
}
```

```
Read Data from File :  
d:\ByteScore.txt  
=====
```

Enter id :	1
Enter name :	EE
Enter score :	58
Enter id :	2
Enter name :	CPE
Enter score :	3.9
Enter id :	-1



โปรแกรมอ่านและแสดงผลข้อมูลจากไฟล์ ByteScore.txt

```
import java.io.*;
class std {
    int i;
    String n;
    double s;
}
public class ByteDataInputTest {
    public static void main(String[] args) {
        File ifile = new File("d:\\ByteScore.txt");
        FileInputStream f_in = null;
        BufferedInputStream b_in = null;
        DataInputStream d_in = null;
        System.out.println("Read Data from File : \n" + ifile);
        System.out.println("=====");
        boolean eof=false;  double sum=0;  int c=0;  std[] x = new std[50];
        try {
            f_in = new FileInputStream(ifile);
            b_in = new BufferedInputStream(f_in);
            d_in = new DataInputStream(b_in);
```

โปรแกรมอ่านและแสดงผลข้อมูลจากไฟล์ ByteScore.txt

```
while (!eof) {  
    x[c] = new std();  
    x[c].i = d_in.readInt();  
    x[c].n = d_in.readUTF();  
    x[c].s = d_in.readDouble();  
    sum = sum+x[c].s;  
    c++;  
}  
}  
catch (EOFException e){  
    eof = true;  
}  
catch (IOException e){  
    System.out.println(e);  
}  
finally {  
    try {  
        if (d_in != null)  
            d_in.close();        double avg =sum/c;
```

โปรแกรมอ่านและแสดงผลข้อมูลจากไฟล์ ByteScore.txt

```
for (int i=0;i<c;i++)
    System.out.printf("%-10d%-10s%-6.2f\n",x[i].i,x[i].n,x[i].s);
System.out.println("=====");
System.out.printf("mean = %.2f\n",avg);
System.out.println("=====");
}
catch (IOException e){
    System.out.println(e);
}
}
}
```

```
Read Data from File :
d:\ByteScore.txt
=====
1          EE          58.00
2          CPE         3.90
=====
mean = 30.95
=====
```

เมื่อเปรียบเทียบกับจัดการข้อมูลแบบ Character Stream ด้วยคลาส PrintWriter และ FileReader พบว่า

- การใช้งานคลาส PrintWriter และ FileReader จะมีขั้นตอนที่ยุ่งยากกว่า เนื่องจากการเขียนเขียนข้อมูลลงไฟล์ทีละบรรทัด และในกรณีที่มีข้อมูลมากกว่า 1 ค่า จะต้องใช้เครื่องหมาย “\t” คั่นระหว่างข้อมูล เมื่ออ่านข้อมูลก็ต้องแยกข้อมูลด้วยเครื่องหมาย “\t” เช่นกันจึงเป็นการจัดการที่ยุ่งยาก
- การจัดการข้อมูลแบบ Character Stream จึงเหมาะกับข้อมูลที่เป็นข้อความ ส่วนการจัดการแบบ Byte Stream เหมาะกับข้อมูลชนิดพื้นฐานที่แตกต่างกัน

การทำงานเกี่ยวกับคุณสมบัติของคลาส File

File อยู่ในแพ็คเกจ java.io จะประกอบด้วยเมธอดที่ช่วยในการทำงานกับ file หรือ directory เช่น absolute path, last modify หรือ ขนาดของไฟล์ เป็นต้น มีรูปแบบการใช้งานคลาส File ดังนี้

```
File filename = new File(Path);
```

โดยที่

path

เป็นชื่อและตำแหน่งที่อยู่ของไฟล์

filename

เป็นออบเจกต์ที่สร้างจากคลาส File

เมธอดที่ควรรู้ในคลาส File

ชื่อเมธอด	คำอธิบาย
canRead()	ตรวจสอบคุณสมบัติของไฟล์ว่าสามารถถูกนำไปเปิดอ่านได้หรือไม่
canWrite()	ตรวจสอบคุณสมบัติของไฟล์ว่าสามารถถูกนำไปเปิดเขียนได้หรือไม่
delete()	ลบไฟล์ โดยคืนค่าเป็นจริงเมื่อไฟล์นั้นมีอยู่จริงและสามารถลบได้สำเร็จ
exist()	ตรวจสอบว่าไฟล์ดังกล่าวมีอยู่หรือไม่
getName()	ให้ค่าเป็นชื่อไฟล์
getParent()	ให้ค่าเป็นพาธหรือไดเรกทอรีของไดเรกทอรีที่เก็บไฟล์
getPath()	ให้ค่าเป็นพาธหรือไดเรกทอรีของไฟล์
isDirectory()	ตรวจสอบออบเจกต์ว่าเป็นไดเรกทอรีหรือไม่
isFile()	ตรวจสอบออบเจกต์ว่าเป็นไฟล์หรือไม่
length()	ให้ค่าเป็นขนาดของไฟล์ ซึ่งมีหน่วยเป็นไบต์
mkdir	ใช้สร้างไดเรกทอรี พร้อมกับคืนค่าเป็นจริงเมื่อสามารถสร้างได้สำเร็จ
renameTo()	ใช้เมื่อต้องการจะย้ายไฟล์ หรือเปลี่ยนชื่อไฟล์ ในกรณีที่เราระบุที่เก็บของไฟล์ใหม่จะหมายถึงเป็นการย้ายไฟล์ แต่ในกรณีที่ไม่วระบุที่เก็บไฟล์ใหม่จะเป็นการเปลี่ยนชื่อไฟล์



โปรแกรมการใช้งานเมธอดในคลาส File

```
import java.io.*;
public class FileMethod {
    public static void main(String[] args) {
        File f = new File("d:\\file\\ByteScore.txt");
        File nf = new File("d:\\file\\NewByteScore.txt");
        if (f.exists()) {
            System.out.println("getName: " + f.getName()+ " is " + f.length() + " bytes.");
            System.out.println("getPath: " + f.getPath());
            System.out.println("getParent: " + f.getParent());
            if (f.canWrite())
                System.out.println(f.getName() + " is writable.");
            if (f.canRead())
                System.out.println(f.getName() + " is readable.");
            if (f.isFile())
                System.out.println(f.getName() + " is a file.");
            else if (f.isDirectory())
                System.out.println(f.getName() + " is a directory.");
            else
                System.out.println("Unknown !!!");
        }
    }
}
```

โปรแกรมการใช้งานเมธอดในคลาส File

```
f.renameTo(nf);
if (nf.exists())
    System.out.println("getName: " + nf.getName()+ " is " + nf.length() + " bytes.");
if (f.exists())
    System.out.println("getName: " + f.getName()+ " is " + f.length() + " bytes.");
else
    System.out.println(f.getName() + " is not found !!!");
nf.delete();
if (nf.exists())
    System.out.println("getName: " + nf.getName()+ " is " + nf.length() + " bytes.");
else
    System.out.println(nf.getName() + " was deleted !!!");
}
else
    System.out.println(f.getName() + " is not exists !!!");
}
}
```

โปรแกรมการใช้งานเมธอดในคลาส File

```
getName: ByteScore.txt is 33 bytes.  
getPath: d:\file\ByteScore.txt  
getParent: d:\file  
ByteScore.txt is writable.  
ByteScore.txt is readable.  
ByteScore.txt is a file.  
getName: NewByteScore.txt is 33 bytes.  
ByteScore.txt is not found !!!  
NewByteScore.txt was deleted !!!
```

